```
                ("last_name", models.CharField(max_length=25)),
            ],
        ),
    ]
```

Why plural?

You may have noticed I keep writing *table*s rather than *table*. Remember that a single `Model` class may result in multiple database tables. The `Promoter` class is a single table with three columns, but any class with a `ManyToManyField` results in both a main table for the model and a relationship table containing the IDs of the related objects. And of course, a model can have more than one many-to-many relationship. Each `Model` class results in a main table plus another table for each many-to-many relationship defined.

The name of the migration file gets automatically generated when you run `make-migrations`. It is both important and unimportant at the same time. The run order of the files gets determined by the contents of `.initial` and `.dependencies`. Although 0001_initial.py implies it is first, it doesn't have to be. You could do something creative and muck with the names of the files, and things would still work if the member values got constructed properly. I've never tried this; I wouldn't recommend it, and although the name isn't important to Django, it really helps a programmer understand what order things happen in. Don't muck with the filenames.

10.1.2 Changing a model

The migration system hasn't been around for the entire lifetime of Django. In the olden days, coders had to manage changes to their models manually. This was painful. Several add-ons got created by a variety of coders to help manage this problem, with the most popular being called South. South became so popular that the Django core team decided to integrate it into the framework.

Consider what you would need to do if the migration framework did not exist. Having just the promoter's name in the database isn't enough information. To track why they're famous, you add another field. The corresponding database table now needs a new column to go with this field. Before the existence of the migration framework, you'd have to write a SQL script to take care of this change. If you forget during development that a `Model` class changed, and don't have a corresponding SQL script, your code would crash. Not only does Django detect changes in your models, but you can also run commands to check whether the state of the database matches the state of the models, helping ensure you haven't forgotten anything when you push your code live.

To see how Django handles the addition of a field, let's add a field. Before doing that, make sure you've put some promoter data in the database using the Django Admin. This data complicates things, and you want to see that complication. Once you've got the data in place, add a `CharField` called `famous_for`. The following listing shows the new version of the `Promoter` class.